Windows OpenVPN Client Test Report

Establishing a Secure VPN for Remote Access

Appendix to Deliverable D3 (VPN Client Configuration Guide)

Contents

1 Purpose and test environment

2 Test summary

3 Test procedure and evidence

4 Observations and limitations

5 Conclusion

1 Purpose and test environment

The D3 guide describes the Linux client in detail and listed Windows as untested. This appendix records a hands-on test of the OpenVPN client on Windows against the lab gateway gw-hq, using the same certificates, tls-crypt key, TOTP multi-factor authentication and access-control policy described in D1 and D2. Evidence IDs [E29] to [E38] refer to the additional screenshots added to the Evidence folder.

Item	Value
Client	Windows 11 laptop (Windows 10.0.26300, amd64), PowerShell 5.1 as administrator
Client software	OpenVPN 2.7.8 with OpenVPN GUI 11.66.0.0 (OpenSSL 3.6.5, DCO 2.8.14) [E29]
Server	gw-hq, OpenVPN instance on UDP 1194, address 192.168.232.131 (VMware VMnet8)
Authentication	Client certificate (CN remote-user) signed by SecureLink-VPN-CA, tls-crypt key, user name vpnuser plus TOTP code (Google Authenticator PAM)
Profile	client.ovpn, copied from the server; cipher AES-256-GCM, auth SHA256, remote-cert-tls server
Time zones	Windows client log is local time (IST, UTC+5:30); server journal is UTC. 22:11 IST equals 16:41 UTC.

2 Test summary

ID	Test	Expected result	Result	Evidence
W1	Install OpenVPN and check version	Version 2.6 or later (D3 requirement)	Pass: 2.7.8	E29
W2	Copy CA, client certificate, key, tls-crypt key and profile	All five files present in the config folder	Pass	E30, E31
W3	Check profile settings	Remote, CA, cert, key, tls-crypt, remote-cert-tls, pull-filter and auth-user-pass present	Pass	E31
W4	Connect with user name and TOTP code	Credentials prompt, tunnel up	Pass	E32, E33
W5	Cryptographic parameters	TLS 1.3, AES-256-GCM, server certificate verified	Pass	E33
W6	Split tunnel and reachability	Only 10.10.1.0/24 routed; 10.10.1.1 reachable	Pass: 0% loss	E33
W7	Access control	10.10.2.1 (branch LAN) not reachable	Pass: 100% loss (blocked as designed)	E33
W8	Wrong TOTP code	AUTH_FAILED on client; rejection logged on server	Pass	E34, E35
W9	Correct code after failure	Connection accepted and logged	Pass	E35
W10	Confidentiality on the wire	Only OpenVPN data packets on UDP 1194; no readable ICMP	Pass	E36, E37
W11	Windows device posture (manual)	Antivirus, real-time protection and firewall profiles enabled	Pass (manual check)	E38

3 Test procedure and evidence

3.1 Check the installed client version (W1)

From an administrator PowerShell window, run the OpenVPN binary with --version. The output shows OpenVPN 2.7.8 with OpenSSL, AEAD ciphers and Windows data-channel offload (DCO), which satisfies the minimum version of 2.6.0 set in D3.

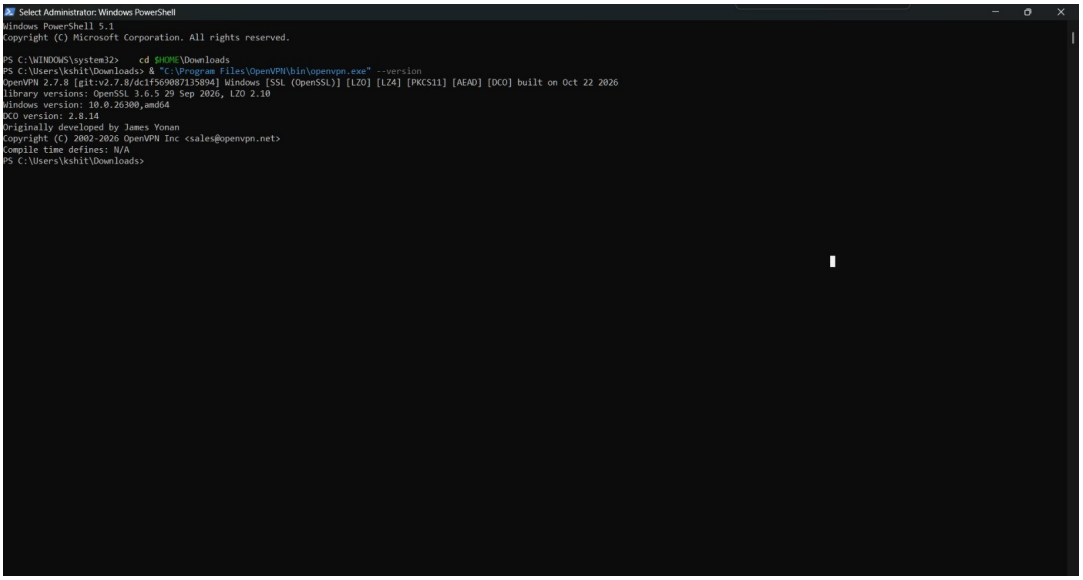


Figure 1. OpenVPN 2.7.8 installed on Windows [E29]

3.2 Copy the certificates and key material (W2)

Create the folder %USERPROFILE%\OpenVPN\config and copy ca.crt, remote-user.crt, remote-user.key and tc.key from the gateway with scp over SSH. The directory listing confirms the four files and their sizes. The private key and tls-crypt key are copied only over an authenticated, encrypted SSH session and are not sent by email or chat.

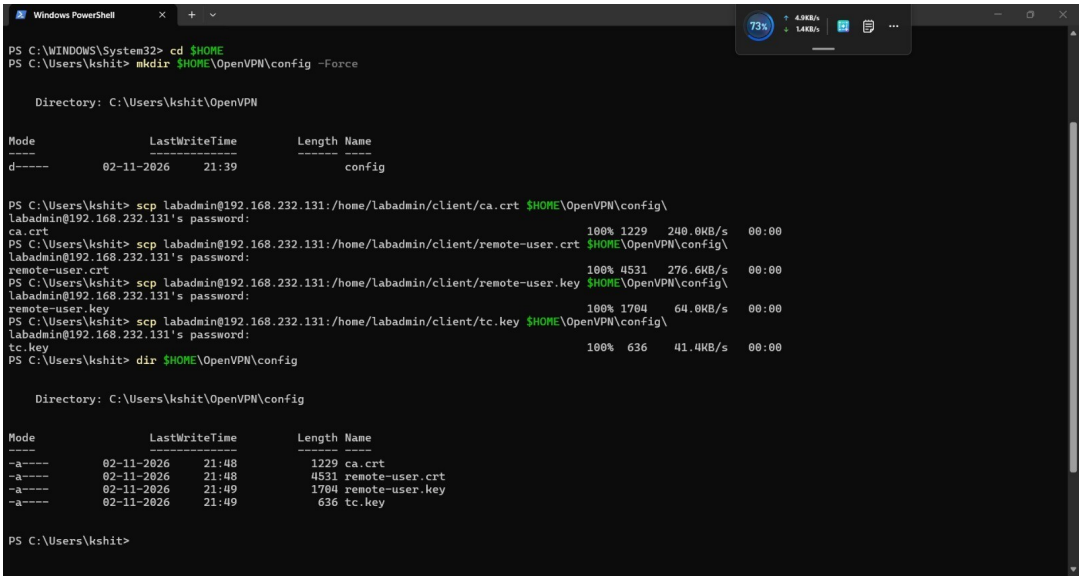


Figure 2. Files copied from gw-hq with scp and listed in the config folder [E30]

3.3 Copy and adjust the client profile (W2, W3)

Copy client.ovpn into the same folder. The profile was created on the Linux client, so the Linux path prefix (/home/kali/) was removed with a PowerShell replace command, after backing up the original as client.ovpn.bak. Select-String then lists the key directives: remote 192.168.232.131 1194, ca, cert, key, tls-crypt, remote-cert-tls server, two pull-filter lines (the gateway is not used as default route) and auth-user-pass.

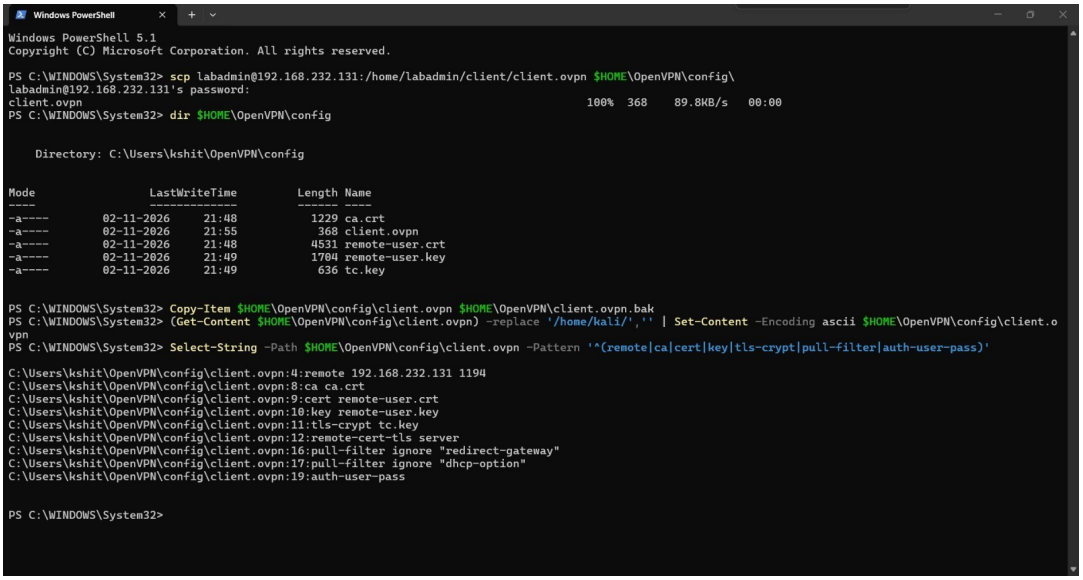


Figure 3. Profile paths corrected for Windows and key directives verified [E31]

3.4 Start the connection and enter credentials (W4)

Right-click the OpenVPN GUI tray icon and choose Connect. The GUI shows Current State: Connecting and prompts for a user name and password. The user name is vpnuser; the password field takes the six-digit TOTP code from the authenticator app, as in the Linux procedure.

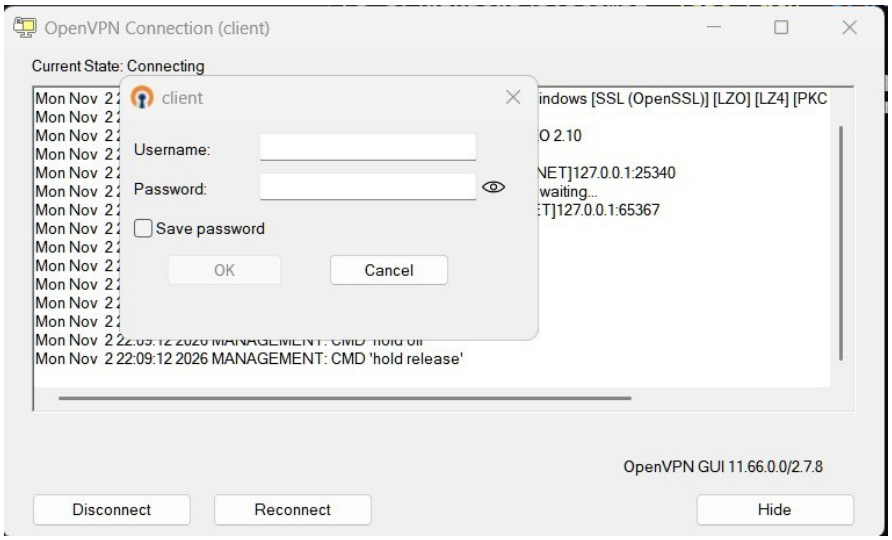


Figure 4. OpenVPN GUI asks for the user name and TOTP code [E32]

3.5 Verify encryption, route and reachability (W5, W6, W7)

After a correct code the log shows the server certificate verified (CN gw-hq, issued by SecureLink-VPN-CA), a TLS 1.3 control channel with TLS_AES_256_GCM_SHA384, and the data channel using AES-256-GCM. The pushed configuration contains a single route, 10.10.1.0/24, and the Windows routing table shows exactly that route through the tunnel address 10.8.0.2. Ping to 10.10.1.1 succeeds with 0% loss. Ping to 10.10.2.1 (the branch LAN) fails with 100% loss: this destination is not routed to remote users and is blocked by the gateway firewall, as designed in D1 and shown on Linux in [E12].

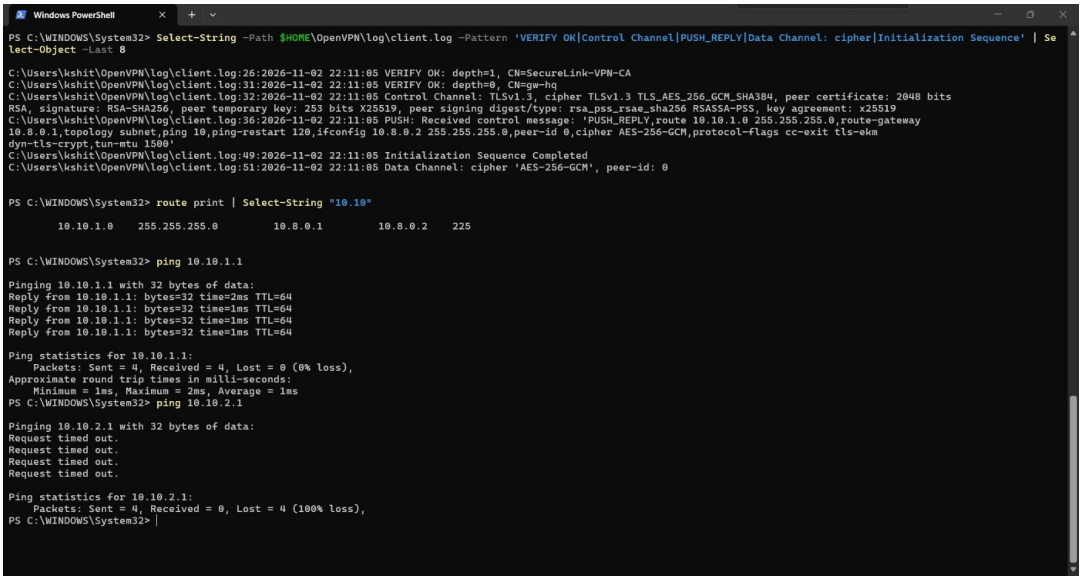


Figure 5. Verified server certificate, TLS 1.3, AES-256-GCM, pushed route, allowed and blocked destinations [E33]

3.6 Negative test: wrong TOTP code (W8)

A wrong code was entered at the prompt. The GUI shows "Wrong credentials. Try again." and the client log contains AUTH: Received control message: AUTH_FAILED. The profile contents shown above the prompt confirm the cipher and digest settings in use.

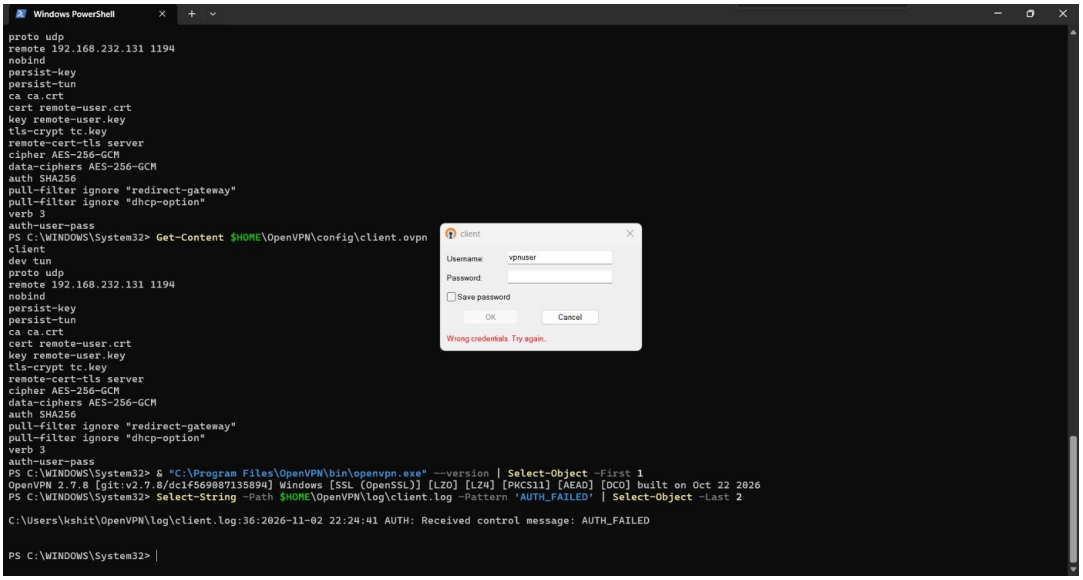


Figure 6. Wrong code rejected: AUTH_FAILED on the Windows client [E34]

3.7 Server-side confirmation (W8, W9)

The gateway journal records the same events from the server side: a first successful login at 16:41:07 UTC, "Invalid verification code for vpnuser" followed by AUTH_FAILED at 16:54:44, and a new successful login at 16:55:47 after the correct code was entered. The source address 192.168.232.1 is the Windows host on the VMnet8 network. The client AUTH_FAILED time (22:24:41 IST, which is 16:54:41 UTC) matches the server entry within a few seconds of clock difference.

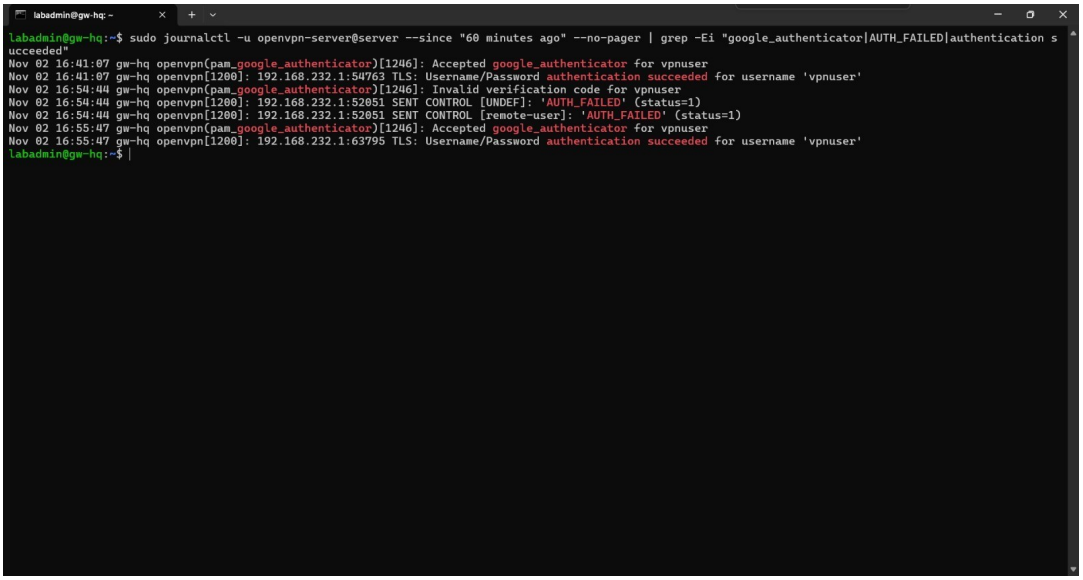


Figure 7. Server journal: accepted codes, one invalid code, AUTH_FAILED sent to the client [E35]

3.8 Packet capture: traffic is encrypted (W10)

Wireshark was run on the VMware VMnet8 adapter while the tunnel was up. With the filter udp.port == 1194, every packet between 192.168.232.1 and 192.168.232.131 is an OpenVPN P_DATA_V2 data packet (the 126-byte packets are the tunnelled ping requests and replies). No application content is visible.

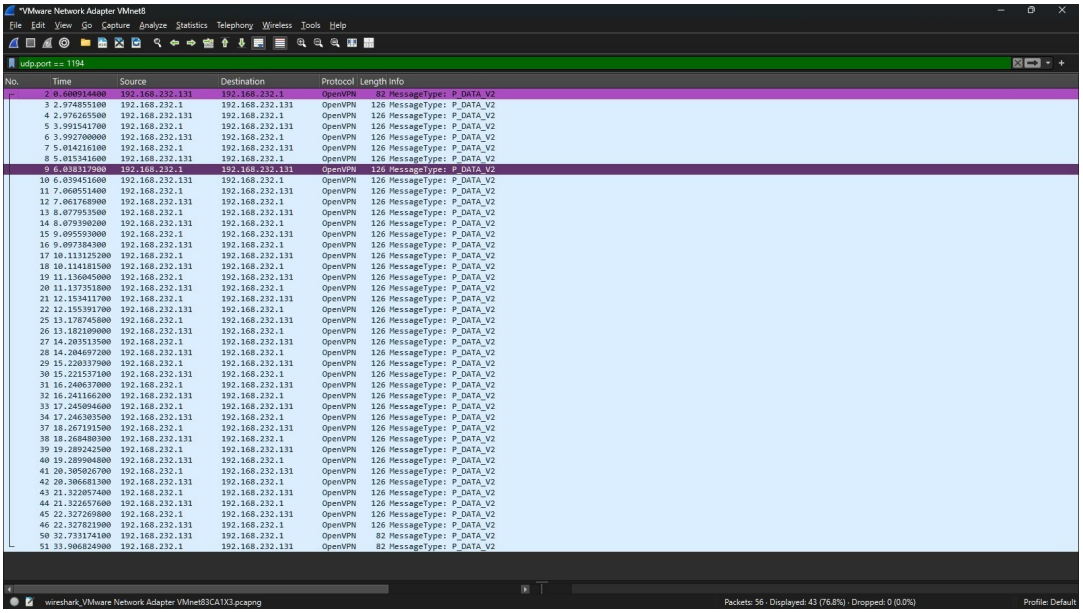


Figure 8. Wireshark: only OpenVPN P_DATA_V2 packets on UDP 1194 [E36]

3.9 Packet capture: no readable ICMP (W10)

Changing the display filter to icmp shows 0 packets displayed out of 56 captured. The pings that crossed the tunnel are inside the encrypted OpenVPN data packets, so an observer on the network sees no ICMP, no inner IP addresses and no payload.

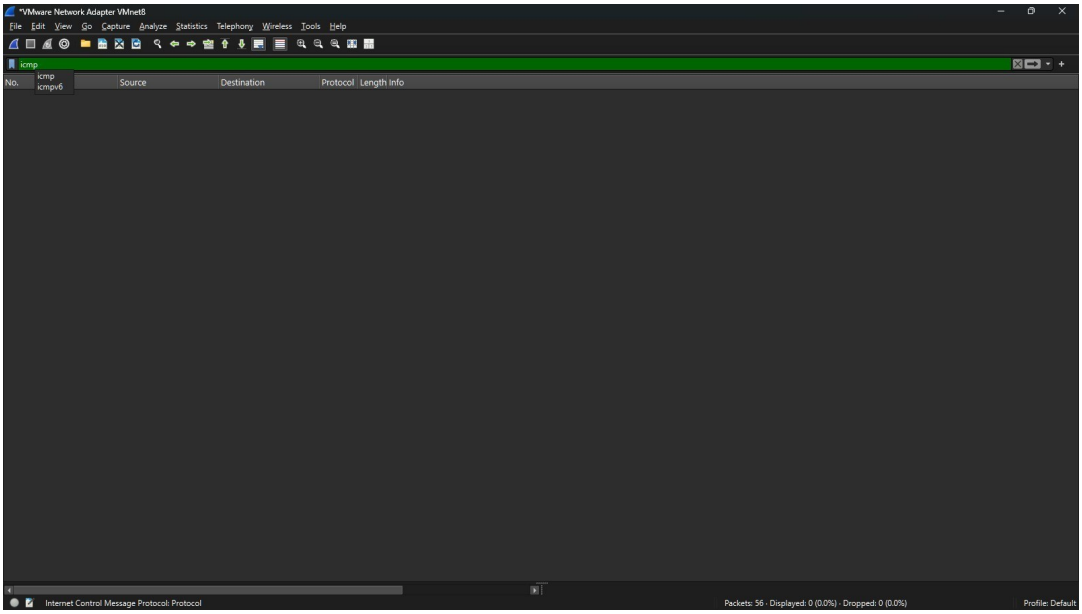


Figure 9. Wireshark filter icmp: 0 packets displayed out of 56 captured [E37]

3.10 Manual Windows posture check (W11)

The Linux client uses the vpn-connect.sh compliance script. No equivalent script was written for Windows, so the same intent was checked by hand with PowerShell: Get-MpComputerStatus reports AntivirusEnabled and RealTimeProtectionEnabled as True, and Get-NetFirewallProfile reports the Domain, Private and Public firewall profiles as enabled. (An earlier attempt to run the firewall command failed because two commands were pasted together on one line; the corrected command is shown.)



Figure 10. Windows Defender and firewall profiles enabled [E38]

4 Observations and limitations

- The Windows client was tested on one machine, with one user account, one certificate and one gateway instance (UDP 1194). The load-balancing profile on instance 2 (UDP 1195) was not repeated on Windows.
- The posture check on Windows is manual. A scripted check (for example a PowerShell wrapper like vpn-connect.sh) would still run on the client and could be bypassed; a server-enforced posture check remains a recommended improvement, as stated in D3.
- The private key sits in a user folder on the laptop. For production, restrict the folder permissions, use a passphrase-protected key or a TPM/PKCS#11 token, and issue one certificate per user.
- The tunnel was tested over a virtual network (VMnet8) on the same host; throughput on Windows was not measured. Performance results in D5 come from the Linux client.
- macOS was not tested. The generic steps in D3 section 6 still apply to it.

5 Conclusion

The OpenVPN client on Windows connected to the gateway using the same certificate, tls-crypt and TOTP controls as the Linux client. The server certificate was verified, TLS 1.3 with AES-256-GCM was negotiated, only the 10.10.1.0/24 route was pushed, the branch LAN was blocked, a wrong TOTP code was rejected and logged, and a packet capture showed only encrypted OpenVPN data on the wire. Windows is therefore recorded as tested for the lab scenario, with the limitations in section 4.